

DiffusionBERT: Improving Generative Masked Language Models with Diffusion Models

Zhengfu He* Tianxiang Sun* Qiong Tang Kuanning Wang
Xuanjing Huang Xipeng Qiu†

School of Computer Science, Fudan University
Shanghai Key Laboratory of Intelligent Information Processing, Fudan University
{zfhel19, txsun19, wangkn20, xjhuang, xpqiu}@fudan.edu.cn
qtang22@m.fudan.edu.cn

Abstract

We present DiffusionBERT, a new generative masked language model based on discrete diffusion models. Diffusion models and many pre-trained language models have a shared training objective, i.e., *denoising*, making it possible to combine the two powerful models and enjoy the best of both worlds. On the one hand, diffusion models offer a promising training strategy that helps improve the generation quality. On the other hand, pre-trained denoising language models (e.g., BERT) can be used as a good initialization that accelerates convergence. We explore training BERT to learn the reverse process of a discrete diffusion process with an absorbing state and elucidate several designs to improve it. First, we propose a new noise schedule for the forward diffusion process that controls the degree of noise added at each step based on the information of each token. Second, we investigate several designs of incorporating the time step into BERT. Experiments on unconditional text generation demonstrate that DiffusionBERT achieves significant improvement over existing diffusion models for text (e.g., D3PM and Diffusion-LM) and previous generative masked language models in terms of perplexity and BLEU score. Promising results in conditional generation tasks show that DiffusionBERT can generate texts of comparable quality and more diverse than a series of established baselines.

1 Introduction

Diffusion models (Sohl-Dickstein et al., 2015; Ho et al., 2020; Song et al., 2021) have recently emerged as a new class of state-of-the-art generative models, achieving high-quality synthesis results on image data (Ramesh et al., 2022; Rombach et al., 2022; Saharia et al., 2022). Though these models captured widespread attention from

* Equal contribution.

† Corresponding author.

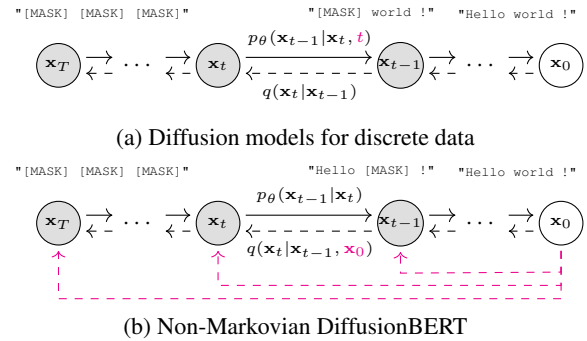


Figure 1: In contrast to conventional discrete diffusion models, DiffusionBERT uses BERT as its backbone to perform text generation. The main differences are highlighted in **color**: (1) DiffusionBERT performs decoding without knowing the current time step while canonical diffusion models are conditioned on time step. (2) The diffusion process of DiffusionBERT is non-Markovian in that it generates noise samples $\mathbf{x}_t$ conditioning not only on $\mathbf{x}_{t-1}$ but also on $\mathbf{x}_0$. Such a non-Markov process is due to our proposed noise schedule.

not only the research community but also the public, applying diffusion models to text data is still challenging and under-explored due to the discrete nature of the text. A few prior works that explored using diffusion models on text data can be divided into two lines. The first is to extend diffusion models to discrete state spaces (Hoogeboom et al., 2021; Austin et al., 2021). The second is to perform the diffusion process and its reverse process in the continuous domain and bridge the continuous and the discrete domain through embedding and rounding (Li et al., 2022; Gong et al., 2022). However, none of these works leveraged pre-trained language models (PLMs, Devlin et al. (2019); Lewis et al. (2020); Raffel et al. (2020); Brown et al. (2020); Qiu et al. (2020)), which are an unmissable treasure in the NLP community.

This work, to our knowledge, is the first attempt to combine diffusion models with PLMs. Such a combination is built upon a shared training ob-

jective between diffusion models and PLMs, i.e., *denoising*. Diffusion models consist of a forward process (data to noise) and a reverse process (noise to data). In the forward process, a small amount of noise is gradually added to the data. Then, a neural network (p_θ in Figure 1) is employed to learn the reverse process step by step, i.e., learn to denoise. Such a denoising neural network is naturally related to a wide class of PLMs that are pre-trained with denoising objectives such as BERT (Devlin et al., 2019) and BART (Lewis et al., 2020). Hence, pre-trained denoising language models can serve as a good starting point to learn the reverse diffusion process. On the other hand, diffusion models also offer a promising training strategy for generative PLMs. In contrast to commonly used generative PLMs (e.g., GPT (Brown et al., 2020)) that rely on an autoregressive factorization of the joint probability, diffusion models provide another way of factorization along the dimension of time and therefore allow the generative model to be not necessarily autoregressive. Thus, diffusion models can be combined with a variety of PLMs that may not be pre-trained for generation.

In the discrete domain, the forward diffusion process can be implemented by a chain of transition matrices that gradually corrupt the clean text. As shown in Figure 1, the clean text "Hello world !" is gradually corrupted into "[MASK] [MASK] [MASK]" during the diffusion process. In this work, we explore using pre-trained denoising language models (e.g., BERT) to learn the reverse diffusion process and demonstrate their advantages in accelerating convergence and improving generation quality. Further, we propose a new noise schedule of the forward process based on the principle of distributing the corrupted information uniformly across the forward process. The noise schedule, called *spindle schedule*, generates noise for $\mathbf{x}_t$ conditioned not only on $\mathbf{x}_{t-1}$ but also on $\mathbf{x}_0$, making the forward process non-Markovian without changing the original training objective. Note that the denoising model takes as input $\mathbf{x}_t$ and time step t to predict $\mathbf{x}_{t-1}$, where t is unseen during the pre-training of language models so we investigate several ways of incorporating the time step into PLMs. As a result, we find that the best result is achieved by throwing away the time information, which we call *time-agnostic decoding* (TAD).

Experimental results on unconditional text generation demonstrate the benefit of combining dif-

fusion models with PLMs: the proposed DiffusionBERT significantly improves the generation quality over existing diffusion models for text generation (e.g., D3PM (Austin et al., 2021) and Diffusion-LM (Li et al., 2022)) and previous generative masked language models (e.g., BERT-Mouth (Wang and Cho, 2019)). DiffusionBERT also matches several strong baselines in conditional generation tasks and shows superior generation diversity. The effectiveness of the proposed spindle schedule and time-agnostic decoding is confirmed by ablation studies. In a nutshell, DiffusionBERT enjoys the best of both worlds.

2 Background

2.1 Diffusion Models

Diffusion models (Sohl-Dickstein et al., 2015; Ho et al., 2020) are a class of latent variable models that are originally designed for continuous domains. A diffusion model is consisting of a forward diffusion process and a reverse diffusion process. Given a sample $\mathbf{x}_0 \sim q(\mathbf{x}_0)$, a Markov chain of latent variables $\mathbf{x}_1, \dots, \mathbf{x}_T$ are produced in the forward process by progressively adding a small amount of Gaussian noise to the sample:

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I}), \quad (1)$$

where $\{\beta_t \in (0, 1)\}_{t=1}^T$ is a noise schedule controlling the step size of adding noise. Eventually $\mathbf{x}_T$ becomes an isotropic Gaussian distribution. If β_t is small enough, the reverse process $q(\mathbf{x}_{t-1}|\mathbf{x}_t)$ is also a Gaussian, which is learned by a parameterized model

$$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t, t) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_\theta(\mathbf{x}_t, t), \Sigma_\theta(\mathbf{x}_t, t)), \quad (2)$$

where $\mu_\theta(\cdot)$ and $\Sigma_\theta(\cdot)$ can be implemented by a U-Net or a Transformer. When conditioning also on $\mathbf{x}_0$, $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$ has a closed form so we can manage to minimize the variational lower bound to optimize $\log p_\theta(\mathbf{x}_0)$:

$$\begin{aligned} \mathcal{L}_{\text{vib}} = & \mathbb{E}_q[D_{\text{KL}}(q(\mathbf{x}_T|\mathbf{x}_0) \parallel p_\theta(\mathbf{x}_T))] \\ & + \mathbb{E}_q\left[\sum_{t=2}^T D_{\text{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t, t))\right] \\ & - \log p_\theta(\mathbf{x}_0|\mathbf{x}_1), \end{aligned} \quad (3)$$

where $\mathbb{E}_q(\cdot)$ denotes the expectation over the joint distribution $q(\mathbf{x}_{0:T})$.

2.2 Diffusion Models in Discrete Domain

For discrete domains, each element of $\mathbf{x}_t$ is a discrete random variables with K categories. For text data, $K = |V|$ is the size of the vocabulary. Denote $\mathbf{x}_t$ as a stack of one-hot vectors, the process of adding noise can be written as

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \text{Cat}(\mathbf{x}_t; \mathbf{p} = \mathbf{x}_{t-1}\mathbf{Q}_t), \quad (4)$$

where $\text{Cat}(\cdot)$ is a category distribution and $\mathbf{Q}_t$ is a transition matrix that is applied to each token in the sequence independently: $[\mathbf{Q}_t]_{i,j} = q(x_t = j|x_{t-1} = i)$. It is easy to obtain that

$$\begin{aligned} q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) &= \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{x}_0)q(\mathbf{x}_{t-1}|\mathbf{x}_0)}{q(\mathbf{x}_t|\mathbf{x}_0)} \\ &= \text{Cat}\left(\mathbf{x}_{t-1}; \mathbf{p} = \frac{\mathbf{x}_t\mathbf{Q}_t^\top \odot \mathbf{x}_0\overline{\mathbf{Q}}_{t-1}}{\mathbf{x}_0\overline{\mathbf{Q}}_t\mathbf{x}_t^\top}\right), \end{aligned} \quad (5)$$

where $\overline{\mathbf{Q}}_t = \mathbf{Q}_1\mathbf{Q}_2 \cdots \mathbf{Q}_t$. Note that $\odot$ is element-wise multiplication and the division is row-wise.

With $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$ at hand, according to Eq. (3), we can use a parameterized model $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t, t)$ to learn the reverse diffusion process.

3 DiffusionBERT

In contrast to recently proposed diffusion models for text, e.g., Diffusion-LM (Li et al., 2022) and DiffuSeq (Gong et al., 2022), which are based on *continuous* diffusion models, we instead explore *discrete* diffusion models to integrate PLMs as the backbone. We first introduce a specific instance of discrete diffusion models (Austin et al., 2021), which considers a transition matrix with an absorbing state for the sake of using PLMs (§ 3.1). Secondly, we introduce a new noise schedule of the forward diffusion process, called spindle schedule, which is based on the principle of distributing the corrupted information uniformly across the forward process (§ 3.2). Then, we investigate several alternatives of incorporating the time step into PLMs for predicting $\mathbf{x}_{t-1}$ given $\mathbf{x}_t$ and t (§ 3.3). Finally, we explore training DiffusionBERT for conditional generation with prompts (§ 3.4).

3.1 Diffusion Models with a Discrete Absorbing State

To be combined with pre-trained denoising language models, we incorporate an *absorbing state*, e.g., [MASK] for BERT, in the Markov process. In

particular, each token in the sequence either stays the same or transitions to [MASK] with some probability. Formally, each entry of the transition matrix at step t is as follows,

$$[\mathbf{Q}_t]_{i,j} = \begin{cases} 1 & \text{if } i = j = [\text{M}], \\ \beta_t & \text{if } j = [\text{M}], i \neq [\text{M}], \\ 1 - \beta_t & \text{if } i = j \neq [\text{M}], \end{cases} \quad (6)$$

where [M] is the abbreviation of [MASK]. Such a Markov process converges to a stationary distribution $q(\mathbf{x}_T)$, which places all probability mass on a sequence with all [MASK] tokens.

The t -step marginal $q(\mathbf{x}_t^i|\mathbf{x}_0^i)$ can be easily obtained in a closed form,

$$q(\mathbf{x}_t^i|\mathbf{x}_0^i) = \begin{cases} \overline{\alpha}_t & \text{if } \mathbf{x}_t^i = \mathbf{x}_0^i, \\ 1 - \overline{\alpha}_t & \text{if } \mathbf{x}_t^i = [\text{M}], \end{cases} \quad (7)$$

where $\overline{\alpha}_t = \prod_{i=1}^t (1 - \beta_i)$, $\mathbf{x}_t^i$ denotes the i -th token in the sequence at step t . Combining with Eq. (3) and (5), we can derive a training objective to optimize $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t, t)$ and generate a sample by performing the reverse diffusion process:

$$p_\theta(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^T p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t, t). \quad (8)$$

3.2 Spindle Noise Schedule

The noise schedule in the continuous domain, such as the linear schedule (Ho et al., 2020) and the cosine schedule (Nichol and Dhariwal, 2021), has shown to be important to the performance of diffusion models.

In contrast to the continuous domain where the noise can be easily controlled by the variance of the Gaussian, (1) *it is less obvious how to control the degree of noise added at each step in the discrete domain*. For the discrete domain, the noise schedule $\beta_t = (T - t + 1)^{-1}$ has been explored for the case of the uniform transition matrix (Sohl-Dickstein et al., 2015; Hoogetboom et al., 2021) and the absorbing-state transition matrix (Austin et al., 2021). However, (2) *such a schedule assumes all tokens carry the same amount of information and does not consider the linguistic difference among the tokens in a sequence*. Besides, (3) *it violates the easy-first-generation nature of denoising language models*. That is, the model tends to generate tokens that are most frequently appearing (and is least surprising) in the training corpus to achieve a

higher likelihood. As the context becomes richer, more details come up in the sequence.

To address the above issues, we consider a noise schedule that (1) measures the added noise at each step by the corrupted information and encourage the corrupted information to be uniformly distributed across the diffusion steps. Since the information is measured independently for each token, (2) different tokens in a sequence are assigned different probabilities of transitioning to the [MASK] token. Moreover, inspired by the easy-first-generation phenomenon, (3) we put the tokens in a sequence in descending order of their information and divide them into T buckets. Each bucket is ensured to contain the same amount of information. That is, we mask the most informative tokens at the start of the forward process and mask the least informative tokens at the end of the forward process such that the learnable reverse process follows an easy-first generative behavior.

In particular, distributing corrupted information uniformly across the forward steps can be formally described by

$$1 - \frac{t}{T} = \frac{\sum_{i=1}^n H(\mathbf{x}_t^i)}{\sum_{i=1}^n H(\mathbf{x}_0^i)} = \frac{\sum_{i=1}^n \bar{\alpha}_t^i H(\mathbf{x}_0^i)}{\sum_{i=1}^n H(\mathbf{x}_0^i)}, \quad (9)$$

where H denotes the entropy, which measures the amount of information of a random variable, $\mathbf{x}_t^i$ denotes the i -th token in the sequence and n denotes the length of the sequence. According to Eq. (7), $\bar{\alpha}_t^i = \prod_{j=1}^t (1 - \beta_j^i)$ denotes the probability that the i -th token remains the same at step t , i.e., $\mathbf{x}_t^i = \mathbf{x}_0^i$. We expect that $\bar{\alpha}_t^i > \bar{\alpha}_t^j$ if $H(\mathbf{x}_t^i) < H(\mathbf{x}_t^j)$ such that easy (low-information) tokens emerges earlier than hard (high-information) tokens during the reverse process. In practice, the entropy of a given token $H(\mathbf{x})$ is calculated by the negative logarithm of its frequency in the training corpus.

Considering these aforementioned properties, we construct $\bar{\alpha}_t^i$ as follows,

$$\bar{\alpha}_t^i = 1 - \frac{t}{T} - S(t) \cdot \tilde{H}(\mathbf{x}_0^i), \quad (10)$$

$$S(t) = \lambda \sin \frac{t\pi}{T}, \quad (11)$$

$$\tilde{H}(\mathbf{x}_0^i) = 1 - \frac{\sum_{j=1}^n H(\mathbf{x}_0^j)}{nH(\mathbf{x}_0^i)}, \quad (12)$$

where $S(t)$ is introduced to control the effect of the informativeness at time step t . It is designed to be sinusoidal to ensure $S(0) = S(T) = 0$ such

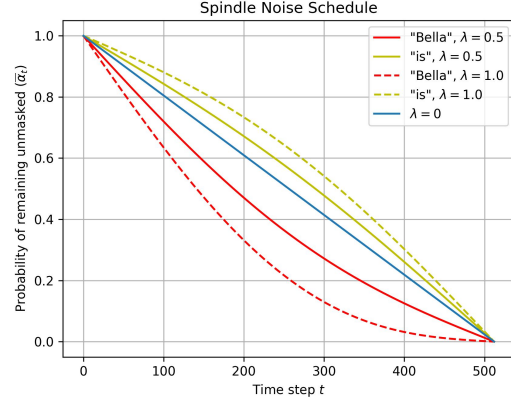


Figure 2: Each token in a sequence has a specific noise schedule depending on how much information is lost when they are masked. For instance, in the sentence "Bella is sitting over there.", "Bella" is the most informative word. Thus it is encouraged to be masked at the early stage so that our model learns to recover it in the last place.

that $\mathbf{x}_t$ can retain all (zero) information when $t = 0$ ($t = T$). The effect of $S(t)$ is controlled by a hyperparameter λ . When $\lambda = 0$, the noise schedule is degraded to $\beta_t = (T - t + 1)^{-1}$ as in Sohl-Dickstein et al. (2015); Hoogeboom et al. (2021); Austin et al. (2021). Figure 2 shows how $\bar{\alpha}$ progresses during the forward process. The schedule is named as *spindle* due to the shape of the probability curves.

In our proposed schedule, the transition probability at time step t depends not only on the current state but also on the original text, making the forward diffusion process non-Markovian. Nevertheless, as revealed by Eq. (5), this does not change the original training objective.

3.3 The Design Space of Feeding Time Steps

Typically, a diffusion model takes as input a noised sample and the time step to predict the denoised sample during the reverse process, i.e., $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t, t)$. However, t is an additional variable that is unseen during the pre-training of language models and therefore it is less trivial how to feed the time information into the PLMs. Here we explore three design choices of feeding time steps.

Layer-wise Time Embedding A straightforward choice is to include the time step as the same way as positional encoding, i.e., using the Transformer sinusoidal embedding or a learnable MLP in each Transformer layer. Note that this way is commonly adopted in previous work (Ho et al., 2020; Austin et al., 2021; Li et al., 2022).

Prefix Time Embedding Prompting language models by prepending trainable soft tokens to the input sequence has shown promising results recently (Lester et al., 2021; Sun et al., 2022). Hence, we also explore including a time step token embedding $\mathbf{v}(t)$ as a prefix of the input token embeddings $\langle \mathbf{v}(\mathbf{x}_t^1), \mathbf{v}(\mathbf{x}_t^2), \dots, \mathbf{v}(\mathbf{x}_t^n) \rangle$. In particular, the time step token is inserted in between the [CLS] token and the input sequence. These added time step token embeddings are trained along with the PLM.

Time-Agnostic Decoding Another alternative is not to explicitly incorporate the time step t because it can be implied by the noised sample $\mathbf{x}_t$. In contrast to the image data, it is easier to implicitly infer the diffusion time step by counting the number of corrupted tokens (i.e., [MASK]) in the noised sequence. In this way, the PLM has to perform iterative decoding while being ignorant of the current time step, i.e., $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$.

3.4 Prompting DiffusionBERT for Conditional Generation

An intriguing property of PLMs is understanding instructions provided in the context and performing desired tasks according to the instructions. Inherited from BERT, DiffusionBERT is also able to solve a wide range of conditional generation tasks by prompting with task descriptions and task texts to be processed (see Appendix.B for examples). To explore the DiffusionBERT for conditional generation, we adopt partial denoising (Gong et al., 2022) to perform the diffusion process conditioning on the partially corrupted text.

4 Experiments

4.1 Tasks and Datasets

We train DiffusionBERT on the One Billion Word dataset (LM1B) (Chelba et al., 2014) for unconditional generation. LM1B is a corpus with about 30 million sentences and a vocabulary of about 793k. For conditional generation, we choose two tasks for evaluating DiffusionBERT, namely Question Generation (QG) and Paraphrasing. Quasar-T (Dhingra et al., 2017) is a Question Answering dataset containing enormous document-question pairs. Gong et al. (2022) constructed a QG dataset from Quasar-T and we follow their data split and task settings. For paraphrasing, we choose Quora Question Pairs (QQP)¹, a widely used question-pairs dataset with

¹<https://www.kaggle.com/c/quora-question-pairs>

147K training samples.

4.2 Baselines

We conduct comparison on unconditional text generation against several non-autoregressive (NAR) baselines: D3PM (Austin et al., 2021), Diffusion-LM (Li et al., 2022), and BERT-Mouth (Wang and Cho, 2019). We consider DiffuSeq (Gong et al., 2022) as a baseline for conditional generation. Several strong baselines reported in their paper are also included for further comparison.

D3PM D3PM is a general framework of discrete diffusion models. We implement an instance of D3PM with the absorbing state and a layer-wise time embedding. Both DiffusionBERT and D3PM are implemented with a sequence length $n = 64$ and diffusion steps $T = 2048$. During inference, we perform DDIM sampling with time step size of 16 in each iteration. Hence, the total inference cost is 128 iterations.

Diffusion-LM Diffusion-LM learns an embedding to map discrete text into the continuous space where it performs Gaussian diffusion process. A rounding step is required to map the continuous embeddings into discrete texts. We re-implemented Diffusion-LM with the model architecture of BERT and diffusion steps $T = 2000$. Since the performance drop of Diffusion-LM is bigger than DiffusionBERT when sampling less steps, we do not skip steps during generation.

BERT-Mouth BERT-Mouth samples text from BERT via order-agnostic autoregressive masked language modeling. Starting from a sequence of [MASK], BERT samples one token at each time step in random order. We continue training BERT on LM1B for fair comparison.

DiffuSeq DiffuSeq introduces a conditional text generation framework for encoder-only diffusion models. It performs diffusion process only on the target (i.e. partial denoising). Experimental results of DiffuSeq show that generation of diffusion models is generally more diverse in conditional generation settings. Such diversity contributes to sample quality with the help of Minimum Bayes Risk (MBR) decoding (Koehn, 2004) i.e., sampling multiple candidates and re-ranking them by their BLEU scores relative to other candidates.

Method	Pretrained	Schedule	Time Step	PPL ↓	BLEU ↑	Self-BLEU ↓
D3PM (Austin et al., 2021)	✗	$(T - t + 1)^{-1}$	LTE	82.34	0.3897	0.2347
			<u>TAD</u>	125.15	0.3390	0.2720
			<u>Spindle</u>	<u>77.50</u>	<u>0.4241</u>	0.2288
Diffusion-LM (Li et al., 2022)	✗	Cosine	LTE	118.62	0.3553	0.2668
	✓	Cosine	LTE	132.12	0.3562	0.2798
BERT-Mouth (Wang and Cho, 2019)	✓	-	-	142.89	0.2867	0.1240
<u>DiffusionBERT</u>	✓	$(T - t + 1)^{-1}$	LTE	92.53	0.3995	0.2118
			<u>PTE</u>	79.95	0.3886	0.2156
			<u>TAD</u>	78.76	0.4213	<u>0.2116</u>
			<u>Spindle</u>	63.78	0.4358	0.2151

Table 1: Main results on LM1B. The methods proposed in this work are marked with wavy lines. The best results are in **bold** and the second best results are underlined. LTE: layer-wise time embedding. PTE: prefix time embedding. TAD: time-agnostic decoding.

4.3 Experimental Setup

In both conditioned and unconditioned settings, our DiffusionBERT is based on BERT-BASE-UNCASED with about 110M parameters. We train DiffusionBERT using the AdamW optimizer (Loshchilov and Hutter, 2019) with learning rate of 3e-6, dropout probability of 0.1 and batch size of 256. We use a 10K-step linear warmup schedule starting from learning rate of 1e-8. For generation efficiency and better alignment with BERT pre-training objective, we use DDIM sampling (Song et al., 2021) in which BERT generates all tokens at first and performs the forward process in Eq. 7 to skip time steps. All experiments are conducted on NVIDIA A100 Tensor Core GPUs. We use 4 GPUs for training and a single GPU for sampling.

To sample from DiffusionBERT trained on LM1B, we use a top- K filter with $K = 30$ and perform 128 steps of inference to align with the settings in Austin et al. (2021).

In the two conditional generation tasks, DiffusionBERT is trained for 100K steps. Sampling involves a top-15 filter and 400 inference steps. We follow Gong et al. (2022) in the MBR size $|\mathcal{S}| = 10$ and sequence length of 128.

4.4 Unconditional Generation

Our main results of unconditional sampling are included in Table 1. We choose BLEU-4 and self-BLEU-4 (Zhu et al., 2018) as the metric for generation quality and diversity, respectively. In particular, we follow Savinov et al. (2022); Caccia

et al. (2020) to sample 1K sentences to compute BLEU score relative to all sentences in the test set. Another 1K sentences are sampled for computing self-BLEU. Overall, DiffusionBERT achieves the best generation quality and diversity trade-off among the considered NAR methods. Besides, the perplexity of DiffusionBERT with the spindle noise schedule is substantially lower. Evidence of lower bound is used as a proxy of the perplexity of DiffusionBERT and D3PM since the exact likelihood of diffusion models is intractable.

DiffusionBERT vs. Other Generative BERT Models We compare DiffusionBERT with another representative generative masked language model, BERT-Mouth (Wang and Cho, 2019). Experimental results show that DiffusionBERT achieves better performance in terms of the perplexity and the BLEU score. We attribute the superior performance of DiffusionBERT to its one-time sampling of all tokens, which helps DiffusionBERT generate more coherent text, especially in a long range. Although such decoding may face the problem of multimodality (Gu et al., 2018), inappropriate phrases can be fixed in the upcoming diffusion steps. The probabilistic modeling offers more flexibility in that generated tokens with low probability are more likely to be masked and re-sampled. Wang and Cho (2019) also proposed to continue masking and predicting tokens after the whole sequence is complete. But such randomness in the selection and replacement of tokens results in low inference speed.

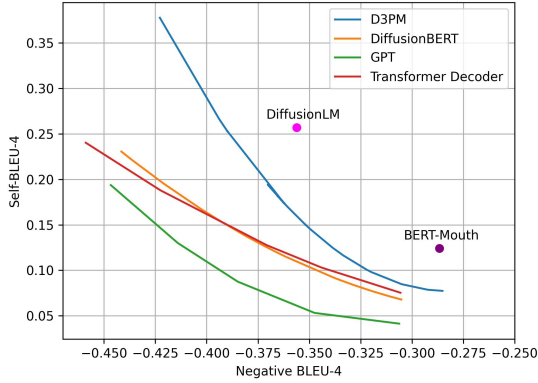


Figure 3: BLEU scores on the LM1B test set. Left is better, lower is better.

Discrete vs. Continuous Diffusion Models We then focus on the comparison of discrete and continuous diffusion models for text generation. To achieve this, we mainly compare DiffusionBERT with Diffusion-LM, which is based on continuous diffusion models. As a result, despite of its outstanding controlling ability, we show that the texts generated by Diffusion-LM have a lower quality than DiffusionBERT. Though both DiffusionBERT and Diffusion-LM adopt the same configuration of Transformer, it is worth noting that the superior performance of DiffusionBERT may be contributed by not only the discrete diffusion models but also the use of pre-trained models. To disentangle the effect of pre-training and discrete/continuous diffusion models, we also explore initializing Diffusion-LM with BERT. As shown in Table 1, training Diffusion-LM from BERT initialization performs even worse than training from scratch. We conjecture that the continuous nature of Diffusion-LM is not compatible with the initialization from BERT since the embedding learned by BERT may not be suitable for the Gaussian diffusion process. In contrast, the comparison of D3PM and DiffusionBERT shows that DiffusionBERT benefits much from the BERT initialization due to its discrete diffusion process.

Effect of Time Step In terms of both likelihood and generation quality, the layer-wise time embedding (LTE) lags far behind the other two time step designs for DiffusionBERT while time-agnostic decoding (TAD) achieves the best result. By contrast, D3PM without time step embedding performs significantly worse. In a nutshell, simplifying time step design has positive effect on Dif-

Method/Metric	Quality		Diversity	
	BLEU $\uparrow$	Rouge-L $\uparrow$	Self-BLEU $\downarrow$	Div-4 $\uparrow$
GRU-attention	0.0651	0.2617	0.9999	0.3178
Transformer-base	0.0364	0.1994	0.8767	0.4055
GPT2-base FT	0.0741	0.2714	0.1403	0.9216
GPT2-large FT	0.1110	0.3215	0.2910	0.8062
GPVAE-T5	<u>0.1251</u>	0.3390	0.3567	0.7282
NAR-LeVT	0.093	0.2893	0.983	0.4776
DiffuSeq	0.1731	0.3665	0.2789	0.8103
DiffusionBERT	0.0971	<u>0.3420</u>	0.0703	0.9372

(a) QG

Method/Metric	Quality		Diversity	
	BLEU $\uparrow$	Rouge-L $\uparrow$	Self-BLEU $\downarrow$	Div-4 $\uparrow$
GRU-attention	0.1894	0.5129	0.9958	0.3287
Transformer-base	0.0580	0.2489	0.7717	0.4312
GPT2-base FT	0.1980	0.5212	0.5480	0.6245
GPT2-large FT	0.2059	0.5415	0.7325	0.5020
GPVAE-T5	0.2409	0.5886	0.5604	0.6169
NAR-LeVT	0.2268	0.5795	0.9995	0.3329
DiffuSeq	<u>0.2413</u>	<u>0.5880</u>	<u>0.2732</u>	<u>0.8641</u>
DiffusionBERT	0.2420	0.5845	0.1504	0.9770

(b) Paraphrase

Table 2: Results on conditional generation tasks. The best and second best results are remarked in **bold** and underlined, respectively.

fusionBERT but is quite harmful for D3PM. This suggests that initializing p_θ with PLMs enables DiffusionBERT to perform generation without explicitly providing time information yet achieving better generation results. The resemblance between BERT pre-training objective and absorbing diffusion models makes it easier for DiffusionBERT to generalize to noisier scenarios while a Transformer encoder trained from scratch needs a specific time-aware module to model the reverse process.

Effect of the Spindle Noise Schedule We try our proposed spindle noise schedule on both DiffusionBERT and D3PM. The perplexity is improved by 5.8% and 19% for D3PM and DiffusionBERT, respectively. Besides, D3PM with the spindle schedule outperforms that with the standard $(T - t + 1)^{-1}$ schedule in terms of BLEU score. The same trend holds for DiffusionBERT but with a smaller margin.

4.5 Quality-Diversity Trade-off

Figure 3 demonstrates the quality-variation trade-off by changing the truncation parameter K or the sampling temperature τ in 10 values², DiffusionBERT exhibits comparable generation ability with

²Except for Diffusion-LM and BERT-Mouth since controlling temperature has little effect.

a Transformer decoder trained from scratch and pushes the Pareto front of NAR generation quality/diversity trade-off by a large margin. However, it still falls behind pre-trained AR models of the same size.

4.6 Training Efficiency

One important feature of DiffusionBERT is that with time-agnostic decoding, all parameters are initialized by pre-trained models. Consequently, the model includes fewer parameters and gets rid of adapting new parameters, improving the training efficiency. We only train DiffusionBERT for 40% steps of D3PM and 20% steps of DiffuSeq till convergence. Appendix.D provides a detailed comparison of convergence speed between DiffusionBERT and the diffusion baselines.

4.7 Conditional Generation

To evaluate the generation quality and diversity, we choose two metrics in each aspect. Besides BLEU and Self-BLEU scores, we also report Rouge-L for quality and Div-4 score for diversity. Higher Rouge-L (resp. Div-4) suggests better generation quality (resp. diversity).

As shown in Table 2, DiffusionBERT achieves competitive performance on both tasks and surpasses other baselines by a large margin in terms of diversity. We attribute such variety to the knowledge BERT obtained during pre-training and the diffusion generative process (Li et al., 2022). Apart from lexical or grammatical diversity, DiffusionBERT covers a wider range of semantic meanings. Moreover, such diversity contributes to generation quality via MBR decoding (Gong et al., 2022).

5 Related Work

5.1 BERT for Text Generation

It has been shown by Wang and Cho (2019) that the transfer-learning ability of BERT does not only helps to achieve impressive results in natural language understanding but also benefits sequential sampling for text generation. However, its bi-directionality nature holds BERT from matching the decoder-only counterparts (Radford et al., 2018) in modeling text from left to right.

5.2 Diffusion Models for Text

This work lies in the line of diffusion models, a latent variable generative framework proposed by

Sohl-Dickstein et al. (2015). It has been architecturally improved by Ho et al. (2020) and has gained broad attention for its impressive generation ability and controllability in image generation (Ramesh et al., 2022; Saharia et al., 2022). Despite that, diffusion models for text still struggle to match autoregressive models in various generation tasks. Since the Gaussian noise proposed in Sohl-Dickstein et al. (2015) cannot be directly applied to discrete data, they also introduced a discrete forward process with a Bernoulli transition kernel. Hooeboom et al. (2021) generalized from Bernoulli to categorical distributions. A more general family of discrete diffusion processes was introduced in Austin et al. (2021); Hooeboom et al. (2022), including absorbing kernels and combinations of absorbing and uniform transition kernels. Li et al. (2022); Gong et al. (2022) models text in the continuous embedding space, which is closer to the settings in earlier works of diffusion models.

5.3 Non-Autoregressive Text Generation

Absorbing discrete diffusion models resembles conditional masked language models (Ghazvininejad et al., 2019) in that both methods predict the whole sequence simultaneously and follows a construct-destruct pattern to iteratively refine the generated text. The difference between those two models has been discussed in Austin et al. (2021). Savinov et al. (2022) proposed to approach the problem of non-autoregressive text modeling via unrolling the generation path, which resembles the idea of diffusion models for unconditional text generation. Non-autoregressive models are also considered in translation but implemented in various ways, e.g., insertion/deletion (Gu et al., 2019) and iterative sequence alignment (Saharia et al., 2020).

6 Conclusion

This work aims to approach the problem of text generation with non-autoregressive models. To achieve this, we combine pre-trained denoising language models with absorbing-state discrete diffusion models. The training procedure of our method includes two main deviations from current discrete diffusion models, i.e., a new family of time step designs and the spindle noise schedule. The spindle noise assigns a schedule for each token according to its frequency in the training corpus. Experimental results of unconditional generation demonstrate the success of DiffusionBERT in terms of genera-